

mdspan: rename pointer and contiguous

Document #: P2604
Date: 2022-06-15
Project: Programming Language C++
LEWG
Reply-to: Christian Trott
<crtrott@sandia.gov>

Contents

- 1 Revision History
 - 1.1 Initial Version 2022-06 Mailing
- 2 Description
 - 2.1 The Issue
 - 2.1.1 pointer
 - 2.1.2 contiguous
- 3 Wording
 - 3.1 pointer to `data_handle_type`
 - 3.2 `data()` to `data_handle()`
 - 3.3 contiguous to exhaustive

§ 1 Revision History

§ 1.1 Initial Version 2022-06 Mailing

§ 2 Description

§ 2.1 The Issue

During LWG review a few members of classes in the `mdspan` proposals were identified as problematic, this paper proposes renaming those members.

§ 2.1.1 pointer

The pointer type is provided by the *accessor policy* of an `mdspan`. While it actually is `accessor_type::element_type*` for the only proposed accessor policy in P0009, the

requirements for pointer are very loose. It does not need be dereferencable, indexable or most any other thing one would expect from a C++ *pointer*. The only thing happening with an `mdspan` pointer is that it is handed together with an offset to the accessor policy's, access function, which returns a *reference*.

To make this clearer we propose to rename `pointer` to `data_handle_type`, and `mdspan::data()` to `mdspan::data_handle()`. `SOME_handle_type` combined with a `SOME_handle()` function to retrieve it has precedence in the library, with `std::thread` which has `native_handle_type` and `native_handle()` member types.

In `mdspan` `pointer` is serving as an opaque handle to data, hence `data_handle`.

`pointer` has uses in the *accessor policy requirements*, the `default_accessor` class, and the `mdspan` class.

§ 2.1.2 contiguous

`contiguous` is used in the `mdspan` proposal to indicate that for a given *layout mapping* `map` for all values in the range `[0, map.required_span_size())` there exists a multidimensional index in the maps index space, that returns that value when used in its mapping operator.

However, the concern was brought up that these values are not necessarily return in a linear order. For example a 1D custom mapping could map the values `{0, 1, 2, 3, 4}` to the offsets `{4, 1, 3, 0, 2}`. This would still be contiguous in the sense it is used in `mdspan`.

To avoid confusing we propose to replace `contiguous` with `exhaustive`, indicating that the entire range of possible return values of the mapping operator, is in fact used by the mapping.

§ 3 Wording

Since P0009 is under active revision in LWG exact wording is not proposed, since there is no up-to-date target.

§ 3.1 pointer to data_handle_type

Every use of `pointer` in the P0009 wording section is to be replaced by `data_handle_type`. Note: there is a use of `is_pointer_v` and `Pointer` in the proposal after merging of P2554, which needs to remain as such (i.e. for the deduction guide in question, it really is a C++ pointer since we have to use `remove_pointer_t` on it).

§ 3.2 data() to data_handle()

The `mdspan::data()` function is renamed to `mdspan::data_handle()`.

§ 3.3 contiguous to exhaustive

Every use of `contiguous` in the wording section would be replaced with `exhaustive`, which specifically renames the `is_contiguous` and `is_always_contiguous` members of mappings (layout mapping requirements, `layout_left`, `layout_right` and `layout_stride`) and the `mdspan` class itself. Furthermore, two uses of *contiguous* in prose text in notes will be replaced.